Guía de Desarrollo: Proyecto RestBase

Pipeline de Machine Learning — Clasificación de Restaurantes

Documento de orientación para equipos de trabajo

Esta guía no contiene código. Su función es orientar el razonamiento, no reemplazarlo.

Contexto del proyecto. La base de datos `restbase`, alojada en el CTU Relational Learning Repository, contiene información de aproximadamente 9.000 restaurantes distribuidos en tres tablas relacionales: `generalinfo`, `location` y `geographic`. El objetivo es construir un clasificador binario que determine si un restaurante debe ser recomendado o no, a partir de la variable `review` transformada en variable binaria.

Principio fundamental del proyecto. Cada decisión técnica debe poder justificarse en términos del problema de negocio. No se acepta "lo hicimos así porque funcionó". Se espera: "lo hicimos así porque... y el efecto esperado es...".

Módulo 1 — Extracción y Exploración con SQL

Este módulo ocurre completamente en SQL, antes de cargar cualquier dato en Python. El objetivo es entender la estructura de la base, detectar problemas y construir la tabla analítica con las features de partida.

1.1 Reconocimiento de la base

P1. ¿Cuántas tablas tiene la base y cuáles son sus llaves primarias y foráneas?

P2. ¿Cuál es la cardinalidad real de `id_restaurant` en cada tabla? ¿Es consistente?

P3. Antes de hacer cualquier `JOIN`, ¿cuántos registros tiene cada tabla por separado?

P4. ¿Qué tipo de relación hay entre `generalinfo` y `location`? ¿Y entre `generalinfo` y `geographic`?

P5. ¿Existe alguna tabla que pueda producir duplicados al hacer el `JOIN`? ¿Por qué?

1.2 Exploración de la variable objetivo

P1. ¿Cuál es el rango de valores de `review`? ¿Qué representa ese rango en términos de negocio?

P2. ¿Cuál es la distribución de `review`? ¿Es simétrica, sesgada, bimodal?

P3. Si se usa la mediana como umbral de binarización, ¿qué proporción de restaurantes queda en cada clase? ¿Es ese balance aceptable para un clasificador?

P4. ¿Qué argumento de negocio sustenta el umbral elegido? ¿Qué implicaría usar un umbral más alto o más bajo?

1.3 Exploración de variables categóricas

P1. ¿Cuántos valores únicos tiene `food_type`? ¿Cuántos tienen frecuencia menor a 5?

P2. ¿Existen valores en `food_type` que representen el mismo concepto con etiquetas diferentes?

P3. ¿Cuántos `label` únicos hay? ¿Cuál es la distribución de frecuencia? ¿Cuántos aparecen solo una vez?

P4. ¿Cuántos valores únicos de `city` hay entre `generalinfo`, `location` y `geographic`? ¿Son consistentes entre tablas?

P5. Para cada variable categórica: ¿el valor del string identifica al restaurante, o describe una propiedad del restaurante? ¿Qué diferencia implica eso para el modelado?

1.4 Construcción de la tabla analítica

P1. Después del `JOIN` completo, ¿cuántos registros resultan? ¿Es ese número consistente con lo esperado? Si hay más o menos, ¿cuál es la causa?

P2. ¿Qué features pueden construirse directamente en SQL sin riesgo de leakage?

P3. ¿Cómo se puede transformar `food_type` de 145 categorías a un número manejable de grupos sin perder información relevante?

P4. ¿Qué información útil puede extraerse de `label` sin codificar los 7.000 valores únicos?

P5. ¿Cómo se construye la variable `recommended` a partir de `review` en SQL?

Atención:
El conteo de registros antes y después de cada `JOIN` es obligatorio. Un `JOIN` silencioso que duplica o elimina filas contamina todo el pipeline posterior sin dejar rastro visible en las métricas de entrenamiento.

Entregable del módulo:

- Script SQL comentado con todos los `JOIN`s y transformaciones.
- Verificación documentada del conteo de registros en cada paso.
- Tabla analítica exportada con las features construidas en SQL.
- Justificación escrita del umbral de binarización elegido.

Módulo 2 — Limpieza y Preprocesamiento en Python

Este módulo opera sobre la tabla analítica exportada desde SQL. El orden de las operaciones no es opcional: el split viene primero, siempre.

2.1 Auditoría inicial

P1. ¿Cuántos valores nulos hay por columna? ¿Son nulos aleatorios o tienen un patrón?

P2. ¿Hay filas duplicadas? ¿En qué columnas?

P3. ¿Las variables numéricas tienen outliers que puedan distorsionar el modelo? ¿Cómo los detectan?

P4. ¿El tipo de dato de cada columna en Python corresponde a lo que se espera semánticamente?

P5. ¿Qué columnas de la tabla analítica *no deben* entrar al modelo bajo ninguna circunstancia? ¿Por qué?

2.2 División del dataset

P1. ¿En qué proporción dividen train, validation y test? ¿Qué criterio usaron?

P2. ¿Por qué el split debe hacerse *antes* de cualquier imputación o codificación?

P3. ¿La variable objetivo está balanceada en los tres conjuntos después del split?

P4. Si estratifican el split por la variable objetivo, ¿qué garantizan con eso?

2.3 Imputación

P1. Para cada columna con nulos: ¿qué estrategia de imputación es apropiada y por qué?

P2. ¿Sobre qué conjunto se calcula el valor de imputación (media, mediana, moda)? ¿Qué ocurre si se calcula sobre el dataset completo?

P3. ¿Cómo se aseguran de que la misma transformación aprendida en train se aplica igual en validation y test?

2.4 Codificación de variables categóricas

P1. ¿Qué variables categóricas quedan después del trabajo en SQL? ¿Cuántas categorías tiene cada una?

P2. ¿Por qué `OneHotEncoder` puede ser problemático con variables de cardinalidad media?

P3. ¿Qué garantiza que el encoder se ajuste solo sobre train y no sobre el dataset completo?

P4. ¿Qué pasa si en validation o test aparece una categoría que no existía en train? ¿Cómo manejan ese caso?

2.5 Escalado

P1. ¿Qué modelos de los que van a usar son sensibles a la escala de las variables?

P2. ¿Cuál es la diferencia entre `StandardScaler` y `MinMaxScaler`? ¿Cuál aplica mejor en este dataset y por qué?

P3. Si escalan, ¿sobre qué conjunto calculan los parámetros de escalado?

Atención:
Cualquier transformación que use información estadística del dataset (media, mediana, frecuencias, parámetros del encoder) debe calcularse exclusivamente sobre el conjunto de entrenamiento. Aplicarla antes del split o sobre el dataset completo introduce leakage y produce métricas optimistas que no se sostienen en producción.

Entregable del módulo:

- Reporte de auditoría: nulos, duplicados y outliers por columna.
- Decisiones de imputación documentadas con justificación.
- Pipeline de preprocesamiento verificado: split → imputar → codificar → escalar.
- Verificación de que train, validation y test no se contaminaron entre sí.

Módulo 3 — Ingeniería de Features

Este módulo evalúa si las variables disponibles son suficientes o si pueden construirse features más informativas antes de entrenar.

3.1 Evaluación de features existentes

P1. ¿Cuál es la correlación entre cada feature numérica y la variable objetivo? ¿Alguna tiene correlación cercana a cero?

P2. ¿Hay features que estén altamente correlacionadas entre sí? ¿Qué problema puede causar eso en regresión logística?

P3. ¿Las features construidas en SQL (grupos de comida, tier de cadena, popularidad) tienen poder discriminativo sobre la variable objetivo? ¿Cómo lo verifican antes de entrenar?

3.2 Construcción de nuevas features

P1. ¿Qué información combinada de dos columnas podría ser más informativa que cada columna por separado?

P2. La columna `city` proviene de tres tablas distintas. ¿Existe inconsistencia entre ellas? ¿Cómo la resuelven?

P3. ¿Tiene sentido construir una feature que combine el tipo de comida con la región geográfica? ¿Por qué sí o por qué no?

P4. ¿Qué features construidas en SQL podrían refinarse o complementarse en Python?

3.3 Selección de features

P1. Si eliminan una feature antes de entrenar, ¿cómo justifican esa decisión?

P2. ¿Qué técnica usan para evaluar la importancia de cada feature? ¿Es la misma para todos los modelos?

P3. ¿Cuántas features entran finalmente al modelo? ¿Ese número es razonable dado el tamaño del dataset?

Entregable del módulo:

- Tabla de features finales con descripción, origen y tipo.
- Análisis de correlación documentado.
- Justificación de las features descartadas.

Módulo 4 — Clasificación

Cada equipo entrena cuatro modelos: Regresión Logística (obligatoria para todos) más tres modelos asignados individualmente, uno por integrante. El flujo de trabajo es el mismo para los cuatro modelos: entrenar sobre `train`, ajustar hiperparámetros observando `validation`, reportar métricas sobre `validation`. El conjunto `test` permanece completamente bloqueado hasta el Módulo 5.

Flujo obligatorio por modelo

Para cada uno de los cuatro modelos deben seguir este orden sin saltarse pasos:

1. Entrenar una versión inicial con los hiperparámetros por defecto sobre `train`.
2. Evaluar esa versión sobre `validation` y registrar las métricas.
3. Explorar manualmente variaciones de hiperparámetros clave, entrenando siempre sobre `train` y midiendo siempre sobre `validation`.
4. Seleccionar la configuración que mejor desempeño tiene sobre `validation` según la métrica principal del equipo.
5. Documentar la configuración final y sus métricas. Esa versión es la que representa al modelo en la comparación del Módulo 5.

Atención:
La exploración de hiperparámetros se hace mirando únicamente `validation`. Si en algún momento una decisión fue tomada observando resultados sobre `test`, ese modelo queda descalificado de la comparación final. `test` no existe hasta el Módulo 5.

Nota sobre búsqueda de hiperparámetros. Existe una técnica llamada *Grid Search* que automatiza la exploración sistemática de combinaciones de hiperparámetros. La idea intuitiva es la misma que aplicarán manualmente: probar todas las combinaciones relevantes de valores, evaluar cada una sobre un conjunto de referencia y quedarse con la mejor. En este proyecto harán eso de forma manual y controlada, lo cual les permite entender exactamente qué está pasando en cada paso.

4.1 Preguntas comunes a todos los modelos

P1. ¿Cuál es la distribución de clases en `train`? ¿Qué accuracy tendría un clasificador que predice siempre la clase mayoritaria? Ese es su baseline trivial: ningún modelo puede quedar por debajo.

P2. ¿Por qué accuracy no es suficiente como métrica principal en este problema?

P3. ¿Qué error es más costoso en el contexto de negocio: recomendar un restaurante malo, o no recomendar uno bueno? ¿Cómo cambia esa respuesta la métrica que priorizan para seleccionar el mejor modelo?

P4. ¿Qué significa ROC-AUC de 0.5? ¿Y de 1.0? ¿Qué rango de valores esperan para este problema y por qué?

P5. ¿Qué información les da la matriz de confusión que no les da el F1?

4.2 Regresión Logística (todos los integrantes)

P1. ¿Qué supuestos hace la regresión logística sobre las features? ¿Se cumplen en este dataset?

P2. ¿Por qué la regresión logística requiere que las variables estén escaladas? ¿Qué ocurre si no se escala?

P3. El hiperparámetro de regularización `C` controla la penalización del modelo. ¿Qué ocurre cuando `C` es muy grande? ¿Y cuando es muy pequeño? ¿Qué valores explorarán?

P4. ¿Cómo interpretan los coeficientes del modelo en términos de las features? ¿Qué features tienen más peso en la decisión?

P5. El umbral de decisión por defecto es 0.5. Dado el análisis de negocio del punto 4.1, ¿tiene sentido mantenerlo o ajustarlo? ¿Cómo lo decidirían observando solo `validation`?

4.3 Modelo asignado — Integrante 1

P1. ¿Cuál es el principio de funcionamiento de este modelo? Descríbanlo sin fórmulas: ¿cómo decide a qué clase asignar una observación?

P2. ¿Es sensible a la escala de las variables? ¿Requiere algún preprocesamiento distinto al de la regresión logística?

P3. ¿Cuáles son los dos o tres hiperparámetros más importantes de este modelo? ¿Qué efecto concreto tiene cada uno sobre el comportamiento del clasificador?

P4. Propongan al menos tres combinaciones de hiperparámetros para explorar. ¿Qué criterio usaron para elegir esos valores y no otros?

P5. ¿Este modelo tiende a sobreajustar? ¿Cómo detectan si está ocurriendo comparando el desempeño en `train` versus `validation`?

4.4 Modelo asignado — Integrante 2

P1. ¿Cuál es el principio de funcionamiento de este modelo? Descríbanlo sin fórmulas: ¿cómo decide a qué clase asignar una observación?

P2. ¿Es sensible a la escala de las variables? ¿Requiere algún preprocesamiento distinto al de la regresión logística?

P3. ¿Cuáles son los dos o tres hiperparámetros más importantes de este modelo? ¿Qué efecto concreto tiene cada uno sobre el comportamiento del clasificador?

P4. Propongan al menos tres combinaciones de hiperparámetros para explorar. ¿Qué criterio usaron para elegir esos valores y no otros?

P5. ¿Este modelo tiende a sobreajustar? ¿Cómo detectan si está ocurriendo comparando el desempeño en `train` versus `validation`?

4.5 Modelo asignado — Integrante 3

P1. ¿Cuál es el principio de funcionamiento de este modelo? Descríbanlo sin fórmulas: ¿cómo decide a qué clase asignar una observación?

P2. ¿Es sensible a la escala de las variables? ¿Requiere algún preprocesamiento distinto al de la regresión logística?

P3. ¿Cuáles son los dos o tres hiperparámetros más importantes de este modelo? ¿Qué efecto concreto tiene cada uno sobre el comportamiento del clasificador?

P4. Propongan al menos tres combinaciones de hiperparámetros para explorar. ¿Qué criterio usaron para elegir esos valores y no otros?

P5. ¿Este modelo tiende a sobreajustar? ¿Cómo detectan si está ocurriendo comparando el desempeño en `train` versus `validation`?

Entregable del módulo:

- Versión final de cada modelo con hiperparámetros documentados y justificados.
- Para cada modelo: tabla de variantes exploradas con sus métricas sobre `validation`.
- Matriz de confusión de cada modelo sobre `validation`.
- Reporte de métricas sobre `validation`: Precision, Recall, F1 y ROC-AUC por modelo y por clase.
- Curva ROC de los cuatro modelos en una misma figura para comparación visual.

Módulo 5 — Comparación de Modelos y Evaluación Final

Este módulo tiene dos etapas claramente separadas. Primero se comparan los cuatro modelos usando exclusivamente `validation` y se selecciona uno. Luego, y solo una vez que la decisión está tomada, se evalúan **los cuatro modelos** sobre `test` para verificar si el ranking observado en `validation` se sostiene.

5.1 Comparación sobre validation y selección del modelo final

Qué deben hacer: construir una tabla comparativa con las métricas de los cuatro modelos sobre `validation`, identificar el modelo ganador con base en la métrica principal definida en el Módulo 4, y documentar esa decisión antes de abrir `test`.

P1. ¿Cuál es la métrica principal de selección? ¿Fue definida antes de ver los resultados o después? ¿Por qué importa ese orden?

P2. Al comparar los cuatro modelos sobre `validation`, ¿el ranking es claro o hay modelos con desempeño muy similar? ¿Qué hacen cuando la diferencia entre dos modelos es marginal?

P3. ¿Las curvas ROC de los cuatro modelos se cruzan? ¿Qué implica un cruce para la decisión de selección?

P4. Si dos modelos tienen métricas similares sobre `validation`, ¿qué otros criterios consideran: interpretabilidad, tiempo de entrenamiento, sensibilidad a hiperparámetros?

P5. ¿Cuál es el modelo seleccionado? Escriban en una sola oración la justificación completa antes de continuar.

Atención:
La selección del modelo final debe quedar registrada y fechada **antes** de evaluar sobre `test`. Cambiar la selección después de ver los resultados sobre `test` es el error metodológico más grave de este módulo.

5.2 Evaluación sobre test

Qué deben hacer: evaluar los cuatro modelos sobre `test` exactamente una vez, sin ningún ajuste posterior. Comparar el ranking obtenido en `test` con el ranking observado en `validation`.

P1. Antes de abrir `test`: ¿qué resultado esperan para el modelo seleccionado? ¿Esperan que las métricas suban, bajen o se mantengan respecto a `validation`? ¿Por qué?

P2. Después de evaluar los cuatro modelos sobre `test`: ¿el modelo que ganó en `validation` también gana en `test`?

P3. Si el ranking cambió entre `validation` y `test`, ¿a qué lo atribuyen? ¿Qué decisiones del pipeline podrían explicar esa diferencia?

P4. ¿Las métricas del modelo seleccionado en `test` son consistentes con las de `validation`? Si hay una caída importante, ¿qué la explica?

P5. ¿Qué conclusión técnica extraen de comparar los cuatro modelos en `test`? ¿El proceso de selección por `validation` fue un buen predictor del desempeño real?

5.3 Reflexión sobre el pipeline completo

P1. ¿Qué decisión del preprocesamiento o de ingeniería de features creen que tuvo mayor impacto en el desempeño final?

P2. ¿Qué información de la base de datos no usaron y que podría haber mejorado el modelo?

P3. ¿El modelo seleccionado es útil en la práctica? ¿Qué necesitaría para funcionar en un sistema real de recomendación?

P4. ¿Qué garantías puede dar el equipo sobre la generalización del modelo a restaurantes de ciudades no vistas durante el entrenamiento?

Entregable del módulo:

- Tabla comparativa de los cuatro modelos con métricas sobre `validation` y sobre `test` en columnas separadas.
- Justificación escrita del modelo seleccionado, redactada antes de abrir `test`.
- Análisis del cambio de ranking entre `validation` y `test`.
- Conclusiones técnicas: mínimo tres párrafos argumentados que conecten las decisiones del pipeline con los resultados finales.
- Cada integrante debe poder responder por cualquier decisión del pipeline en la sustentación individual, independientemente de quién la implementó.

Módulo 6 — Presentación de Resultados

Este módulo cierra el proyecto con una exposición formal de 15 minutos ante una audiencia compuesta por estudiantes, profesores e invitados de otras áreas. Se asignan 5 minutos adicionales para preguntas.

El registro es profesional. El equipo presenta como si sus resultados fueran a informar una decisión real: la dirección de la empresa quiere saber si el modelo es confiable, qué tan bien predice y si vale la pena desplegarlo. Se asume que la audiencia puede hacer preguntas técnicas.

6.1 Estructura de la presentación

Los 15 minutos se distribuyen en cinco bloques. La distribución no es una sugerencia: una presentación que llega al bloque de resultados con dos minutos disponibles ha fallado antes de mostrar lo más importante.

Bloque	Contenido	Tiempo
1	Contexto del problema y objetivo de negocio	2 min
2	Pipeline: decisiones clave de preprocesamiento	3 min
3	Comparación de modelos sobre validation	5 min
4	Resultados sobre test y modelo recomendado	3 min
5	Conclusiones y limitaciones	2 min

6.2 Contenido por bloque

Bloque 1 — Contexto del problema (2 min). La audiencia no conoce la base de datos. El equipo debe establecer en dos minutos qué problema se está resolviendo, qué datos se usaron y qué significa la variable objetivo en términos de negocio. No se presentan tablas relacionales ni esquemas SQL. Se presenta la pregunta: *¿este restaurante merece ser recomendado?*

P1. ¿Pueden explicar el problema sin usar jerga técnica en los primeros treinta segundos?

P2. ¿La definición de la variable binaria está justificada en términos de negocio, o solo en términos estadísticos?

P3. ¿Saben cuántos restaurantes hay en el dataset y qué proporción resulta recomendada según su umbral? ¿Ese número tiene sentido para un cliente?

Bloque 2 — Pipeline y decisiones de preprocesamiento (3 min). No se presenta todo el preprocesamiento: se presentan las tres decisiones que más impactaron la calidad del modelo y por qué se tomaron. La audiencia técnica evaluará si esas decisiones fueron justificadas. La audiencia no técnica evaluará si el equipo sabe qué hizo y por qué.

P1. ¿Cuáles son las tres decisiones de preprocesamiento que más impactaron el resultado final? ¿Pueden argumentarlas en términos de su efecto sobre el modelo?

P2. ¿Cómo presentan el tratamiento de `food_type` y `label` sin perder a la audiencia no técnica?

P3. Si alguien pregunta por qué no usaron `food_type` directamente, ¿tienen una respuesta en menos de treinta segundos?

P4. ¿Pueden mostrar en una sola diapositiva el flujo completo del pipeline sin que parezca un diagrama de ingeniería de software?

Bloque 3 — Comparación de modelos sobre validation (5 min). Este es el bloque central. Cada integrante presenta brevemente su modelo asignado: qué es, cómo funciona en términos intuitivos y cuál fue su mejor configuración. Luego el equipo presenta la comparación conjunta. No se leen métricas de una tabla: se interpretan.

P1. ¿Cada integrante puede explicar su modelo en dos oraciones sin fórmulas, de forma que un gerente lo entienda?

P2. ¿La tabla comparativa muestra solo las métricas relevantes o tiene más columnas de las que la audiencia puede procesar en treinta segundos?

P3. ¿La curva ROC está bien etiquetada y se puede leer a distancia? ¿Saben explicar qué significa el área bajo la curva para alguien que no conoce el concepto?

P4. ¿La métrica principal de selección está definida antes de mostrar los resultados, o parece que la eligieron después de ver quién ganaba?

P5. Si dos modelos tienen desempeño similar, ¿tienen un argumento preparado para justificar cuál prefieren más allá de las métricas?

Bloque 4 — Resultados sobre test y modelo recomendado (3 min). Se presentan los resultados de los cuatro modelos sobre `test` y se declara el modelo recomendado con su justificación. Si el ranking entre `validation` y `test` cambió, eso se menciona y se explica: no se oculta.

P1. ¿El equipo puede declarar claramente cuál es el modelo recomendado y por qué, en menos de un minuto?

P2. Si el modelo que ganó en `validation` no ganó en `test`, ¿tienen preparada una explicación honesta de por qué ocurrió?

P3. ¿Las métricas finales sobre `test` se presentan con contexto? Un F1 de 0.72 no significa nada sin saber cuál era el baseline.

P4. ¿Pueden relacionar el desempeño final con el problema de negocio? ¿Qué porcentaje de restaurantes malos quedaría recomendado con este modelo?

Bloque 5 — Conclusiones y limitaciones (2 min). El cierre no es un resumen de lo que ya se mostró. Es una evaluación honesta: qué funcionó, qué no, y qué haría falta para que este modelo sea útil en un entorno real. Un cliente que escucha limitaciones reconocidas confía más en los resultados que uno que recibe solo buenas noticias.

P1. ¿Las conclusiones son declaraciones argumentadas o simplemente repeticiones de las métricas?

P2. ¿El equipo puede nombrar al menos una limitación real del modelo sin que parezca una autocrítica defensiva?

P3. ¿Qué necesitaría este modelo para ser desplegado en producción? ¿Tienen una respuesta concreta?

P4. ¿La última diapositiva deja clara cuál es la recomendación final del equipo, sin ambigüedad?

6.3 Preparación para preguntas

Los 5 minutos de preguntas son parte de la evaluación. Una respuesta evasiva o imprecisa pesa igual que un error en las métricas. A continuación se listan preguntas probables agrupadas por origen.

Preguntas sobre el problema y los datos.

P1. ¿Por qué usaron la mediana como umbral? ¿Qué pasa si el cliente quiere recomendar solo el 20 % mejor de los restaurantes?

P2. ¿Cómo trataron los restaurantes que pertenecen a cadenas? ¿Eso afecta la predicción para locales independientes?

P3. ¿Cuántos registros tiene el conjunto de test? ¿Es representativo del universo de restaurantes?

Preguntas sobre el preprocesamiento.

P1. ¿Cómo garantizan que no hubo fuga de información entre train y test?

P2. ¿Por qué agruparon los tipos de comida de esa forma y no de otra?

P3. ¿Qué habrían hecho diferente en el preprocesamiento si tuvieran más tiempo?

Preguntas sobre los modelos.

P1. ¿Cómo eligieron los hiperparámetros? ¿Probaron todas las combinaciones posibles o hubo un criterio?

P2. ¿Por qué el modelo que ganó en validación no ganó en test? (Si aplica.)

P3. ¿Cuál de los cuatro modelos recomendarían si el tiempo de predicción fuera un factor crítico?

P4. ¿Cómo saben que el modelo no está sobreajustado?

Preguntas sobre el negocio.

P1. ¿Con este modelo, si un restaurante recibe la etiqueta "recomendado", qué tan probable es que realmente lo sea?

P2. ¿Cambiaría su recomendación si el costo de recomendar un restaurante malo fuera diez veces mayor que el de no recomendar uno bueno?

P3. ¿Este modelo funcionaría igual con restaurantes de una ciudad que no estaba en el dataset de entrenamiento?

6.4 Criterios de evaluación de la exposición

Criterio	Descripción	Peso
Claridad del problema	El problema y la variable objetivo están explicados en términos de negocio, no solo técnicos.	15 %
Justificación del pipeline	Las decisiones de preprocesamiento tienen argumento explícito. No se presentan como pasos mecánicos.	20 %
Comparación de modelos	La comparación es rigurosa: métrica definida antes de ver resultados, visualizaciones legibles, interpretación correcta.	25 %
Resultado final	El modelo recomendado está justificado. Los resultados sobre test se presentan con honestidad, incluyendo discrepancias con validation.	20 %
Respuesta a preguntas	Las respuestas son precisas, honestas y conectadas con el trabajo realizado. No se especula sobre lo que no se hizo.	20 %

Atención:
Responder "no lo hicimos" es aceptable si va seguido de "y la razón es…" o "lo que hicimos en cambio fue…". Inventar una respuesta para no quedar mal es el único error que no tiene recuperación en una presentación profesional.

Entregable del módulo:

- Presentación de diapositivas: máximo 15 láminas para 15 minutos.
- Diapositiva de comparación de modelos con tabla de métricas sobre `validation` y `test` en una sola vista.
- Curva ROC de los cuatro modelos en una figura.
- Declaración explícita del modelo recomendado con su justificación.
- Entrega de los scripts correspondientes a cada uno de los módulos del proyecto.